一、问题分析

本实验旨在实现一个程序，该程序可以接收中缀算术表达式作为输入，并将其转换为后缀表达式，然后对后缀表达式进行计算。表达式中的运算数限定为小于 10 的自然数，且只考虑二元运算符"+"、"-"、"*"、"/"。此外，每个表达式以"="结束，当仅输入"="时，程序终止。

二、总体设计

程序设计分为两个主要部分：

1. **中缀转后缀**：使用一个栈来保存运算符，遍历中缀表达式的每个字符，根据不同的字符类型（数字、运算符、括号）执行相应的操作，最终生成后缀表达式。
2. **后缀表达式求值**：利用栈结构，依次处理后缀表达式的每个元素，遇到数字则入栈，遇到运算符则弹出相应数量的操作数，计算结果并压回栈中，直到处理完所有元素，栈顶即为表达式的值。

三、详细设计

3.1 数据结构选择

- `stack<double> nums`：用于存储运算数或中间结果。
- `string postfix`：用来构建和存储转换后的后缀表达式字符串。
- `stack<char> operate`：用于暂时存放运算符，以便按照运算优先级顺序重新排列到后缀表达式中。

3.2 函数功能说明

- `bool isNum(char c)`：判断字符是否为数字或小数点。
- `bool isOp(char c)`：判断字符是否为指定的运算符。
- `int level(char c)`：返回运算符的优先级。
- `int cal(char op, int a, int b)`：执行基本的四则运算，并处理除零错误。
- `void translate(string &s)`：将中缀表达式转换为后缀表达式。
- `int calculate()`：对后缀表达式进行求值。

3.3 算法逻辑描述

- **中缀转后缀**：遍历输入字符串，对于数字直接添加到后缀表达式中；对于运算符，比较其与栈顶运算符的优先级，若当前运算符优先级不高于

栈顶，则持续弹出栈顶运算符直至条件满足，之后将当前运算符压入栈中；遇到左括号无条件压入栈，遇到右括号则持续弹出栈顶运算符直到遇到左括号；最后，将剩余在栈中的运算符全部弹出并添加到后缀表达式末尾。

- **后缀求值**：遍历后缀表达式，遇到数字压入栈中，遇到运算符则从栈中弹出相应数量的操作数，计算结果后压回栈中。整个表达式处理完毕后，栈顶元素即为最终结果。

3.4 边界情况处理

- 对于除法，检查除数是否为 0，防止出现除零错误。
- 在读取输入时，如果输入仅为“=”，则立即退出循环，停止接受新表达式。

四、调试分析

在调试过程中发现的主要问题是关于运算符优先级的处理以及括号的匹配。通过调整算法逻辑，确保了正确的运算顺序。另外，针对后缀表达式的构造，需要特别注意符号间的空格和表达式结尾的“=”处理。

五、用户手册

1. **启动程序**：编译并运行程序。
2. **输入表达式**：每次输入一行包含中缀表达式的文本，表达式应以“=”结束。
3. **查看输出**：每输入一个表达式，程序会输出两行内容，第一行为对应的后缀表达式，第二行为计算的结果。
4. **结束程序**：仅输入“=”即可结束程序运行。

六、测试结果

通过对多个样例的测试，包括但不限于提供的示例，程序均能正确地将中缀表达式转换为后缀表达式，并准确计算出结果。例如：

- 输入：`9+(3-1)*3+1/2=` 输出：
  - 后缀表达式：`931-3*+12/+`
  - 结果：`15`

- 输入：1+2= 输出：
    - 后缀表达式：12+
    - 结果：3

程序成功实现了预期的功能，能够稳定处理多组数据并给出正确答案。

附录-程序

```
// main.cpp 文件

#include <iostream>
#include <string>
#include <vector>
#include <list>
#include <set>
#include <map>
#include <queue>
#include <algorithm>
#include <stack>
#include <iomanip>
using namespace std;


stack<double> nums;
string postfix;
stack<char> operate;

bool isNum(char c){
	if(c>='0'&&c<='9'||c=='.') return true;
	return false;
}

bool isOp(char c){
	return c=='+'||c=='-'||c=='*'||c=='/';
}

int level(char c){
	if(c=='+'||c=='-') return 1;
	if(c=='*'||c=='/') return 2;
	return 0;
}

int cal(char op,int a,int b){
	switch (op) {
		case '+': return a+b;
		case '-': return a-b;
		case '*': return a*b;
		case '/': if (b == 0) {
				cerr << "Error: Division by zero" << endl;
```

```
                exit(1);
            }
            return a / b;
        default: cerr << "Error: Invalid operator" << endl;
            exit(1);
    }
}


void translate(string &s){
    for(auto &subC:s){
        if(subC=='='){
            break;
        }
        if(subC==' ')
            continue;

        if(isNum(subC)){
            postfix+=subC;
            continue;
        }
        if(isOp(subC)){
            if(!operate.empty()){
                char top=operate.top();
                while(level(top)>= level(subC)){
                    postfix+=top;
                    operate.pop();
                    if(operate.empty())
                        break;
                    top=operate.top();
                }
            }
            operate.push(subC);
            continue;
        }

        if(subC=='('){
            operate.push(subC);
            continue;
        }

        if(subC==')'){
            char top=operate.top();
            while(top!='('){
```

```
                    postfix+= top;
                    operate.pop();
                    top=operate.top();
                }
                operate.pop();
            }
        }
        while(!operate.empty()){
            postfix+=operate.top();
            operate.pop();
        }

}

int calculate(){
    stack<int> ans;
    for(auto &subC:postfix){
        if(isNum(subC))
            ans.push(subC-'0');
        else{
            int numb = ans.top();   ans.pop();
            int numa = ans.top();   ans.pop();
            ans.push(cal(subC,numa,numb));
        }
    }
    return ans.top();
}

int main()
{
    string input;
    cout<<fixed<<setprecision(0);
    while(true){
        cin>>input;
        if(input=="=")
            break;
        translate(input);
        cout<<postfix<<endl<<calculate()<<endl;
        postfix.clear();
    }
    return 0;
}
```